

Rapport de TIPE

Implémenter un réseau pair à pair à l'aide de b-couplages stables

S Roux, MPI, 2025-2026

5 juin 2026

Table des matières

1	Introduction	1
2	Implémentation d'un algorithme de calcul de b-couplage stable dans un réseau dynamique	2
2.1	Calcul d'un b -couplage stable dans un réseau statique	2
2.2	Adaptation dynamique naïve de l'algorithme de calcul	4
2.3	Adaptation dynamique de l'algorithme de calcul par modification locale	4
3	S'affranchir de la condition d'acyclicité	6
4	Conclusions et perspectives	8
5	Annexes	8
5.1	Preuve de terminaison de <i>STATIQUE</i>	8
5.2	Code principal en Ocaml	9

1 Introduction

Les réseaux pair à pair permettent le partage de fichiers de manière décentralisée, où chaque utilisateur se comporte comme un serveur, capable de partager les fichiers qu'il possède [1]. Chaque utilisateur ne peut partager et recevoir qu'un nombre limité de fichiers à la fois, ce qui mène à prioriser certains transferts plutôt que d'autres. Une problématique émerge donc : comment optimiser, à chaque instant, le transfert des segments du fichier (origine et destination), tout en garantissant un temps de convergence algorithmique suffisamment faible (soit inférieur au temps caractéristique d'évolution du réseau) ?

Un *réseau* désignera par la suite un ensemble R d'utilisateurs, nommés *noeuds*. Pour chaque noeuds $(n, p) \in R^2$, $\sigma(n, p)$ désigne un nombre positif nommé la *marque de p selon n* . La liste $Pref(n)$, nommée *liste de préférence*, ordonne l'ensemble des autres noeuds du réseau par marque croissante. R est dit *statique* s'il est constant au cours du temps, et au contraire *dynamique* si des noeuds peuvent y être ajoutés et retirés. On nomme *graphe de préférence de R* le graphe orienté dont les sommets sont les noeuds de R et où une unique arête pointe de chaque noeud n vers le noeud p de marque maximale :

$$\sigma(n, p) = \max_q \sigma(n, q) \tag{1}$$

Notons que chaque noeud y est de degré sortant 1. On dira qu'un graphe de préférence est cyclique s'il contient un cycle de longueur au moins 3.

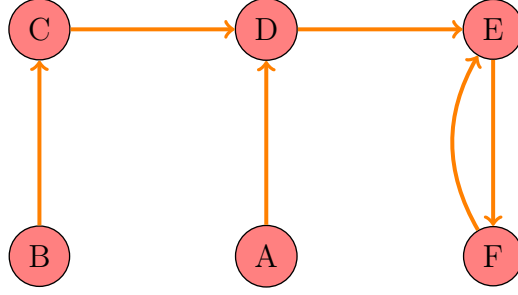


FIGURE 1 – Graphe de préférence *acyclique* associé à un réseau $\{A, B, C, D, E, F, G\}$

Pour $b \in \mathbb{N}^*$, un b -couplage sur un réseau R désigne une partie $C \in R^2$ telle que

$$\forall n \in N, \#\{\{n, p\} | p \in N\} \leq b \quad (2)$$

Ce b -couplage C est dit *stable* s'il est de cardinal maximal et si

$$\forall \{n, p\} \in C, \forall \{n', p'\} \in N^2, \sigma(n, p) > \sigma(n, p') \text{ ou } \sigma(p, n) > \sigma(p, n') \quad (3)$$

En l'absence d'ambiguïté, on appellera C un *couplage*.

Le choix des transferts de données à effectuer peut être vu comme un problème de couplage stable dans le réseau formé par l'ensemble des utilisateurs [2]. La marque $\sigma(n, p)$ y représente l'attractivité de p selon n , qui peut par exemple dépendre de la connexion de p et des paquets dont p dispose. Intuitivement, un b -couplage stable contient pour chaque noeud ses arêtes de plus grande marque, en essayant ainsi de satisfaire un maximum de noeuds.

On commence par implémenter (Section 2.1) un algorithme de calcul de b -couplage dans un réseau statique. Deux problématiques théoriques émergent ensuite lors de l'implémentation d'un réseau par des couplages stables. Premièrement, il s'agit d'étendre les algorithmes de calcul de b -couplage aux réseaux dynamiques. Naïvement, il est possible de recalculer un couplage après toute modification (Section 2.2). Toutefois le manque de pertinence algorithmique de cette solution (qu'on nommera *DYNA1*) invite à considérer une extension plus réfléchie s'appuyant sur une modification locale du couplage (Section 2.3) qu'on nommera *DYNA2*. *DYNA1* servira néanmoins à quantifier les performances algorithmiques de *DYNA2*. Deuxièmement, l'existence d'un b -couplage stable n'est pas garantie dans tout réseau ; néanmoins, Matthieu (2007) montre qu'un b -couplage stable existe systématiquement dans un réseau dont le graphe de préférence est acyclique, ce qui est possible si par exemple la fonction σ est symétrique et injective. Certains critères de préférence peuvent toutefois donner naissance à des graphes de préférence cycliques. La détermination systématique d'un couplage stable étant impossible, on s'intéressera en Section 3 à la possibilité de déterminer une approximation de couplage stable.

2 Implémentation d'un algorithme de calcul de b -couplage stable dans un réseau dynamique

2.1 Calcul d'un b -couplage stable dans un réseau statique

On fixe dans la suite un réseau statique R et un entier $b \in \mathbb{N}^*$. Montrons un théorème [2] annoncé en introduction :

Théorème : Si σ est symétrique et injective, alors le graphe de préférence G de R ne contient pas de cycle de longueur au moins 3.

Démonstration : On suppose par l'absurde l'existence d'un cycle $(n_0, n_1, \dots, n_p)$ dans G . Soit $i \in \llbracket 2, n-1 \rrbracket$. Par construction,

$$\sigma(n_i, n_{i+1}) = \max_p \sigma(n_i, p) > \sigma(n_i, n_{i-1}) = \sigma(n_{i-1}, n_i) \quad (4)$$

L'inégalité est stricte par injectivité de σ . On obtient donc que le cycle est strictement croissant selon σ , ce qui est absurde car alors

$$\sigma(n_0, n_1) < \dots < \sigma(n_p, n_0) < \sigma(n_0, n_1) \quad (5)$$

□

Du moment que chaque utilisateur X classe chaque autre Y selon un critère qui donne le même rôle à X et Y (comme la bande passante entre X et Y), la fonction de préférence sera symétrique. Cela justifie de se restreindre à de telles fonctions. Dans la suite de cette partie, on supposera donc que le graphe de préférence de $|R|$ est acyclique.

En complément de ce théorème, on peut montrer qu'un b -couplage stable existe dès lors que G est acyclique [3]. Par soucis de concision, la démonstration n'est pas développée ici.

On implémente un algorithme de calcul de b -couplage stable inspiré par l'algorithme de Gale et Shapley [5]. On le nommera *STATIQUE*(R, b). On définit une *paire bloquante* comme une paire $\{n, p\}$ de noeuds non couplés telle que, soit n ou p n'est pas complètement couplé, soit n et p préfèrent se coupler ensemble plutôt qu'avec le noeud de pire marque avec lequel chacun est couplé.

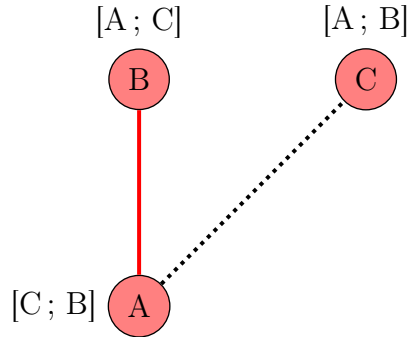


FIGURE 2 – Réseau $\{A, B, C\}$. A côté de chaque noeud est représenté une liste de préférence. Une liaison rouge représente un couple dans le 1-couplage. La ligne en pointillés noire représente le couple que préféreraient former A et C. $\{A, C\}$ est une paire bloquante.

Par définition, un couplage est stable si et seulement si il n'existe pas de paire bloquante. On en déduit alors l'algorithme *STATIQUE*, consistant à retirer une à une les paires bloquantes.

D'après ce qui précède, *STATIQUE* construit bien un couplage stable.

La preuve de terminaison de *STATIQUE* ainsi que certaines précisions sur les choix d'implémentation sont développés en annexe 4.1.

Les figures 3 à 7 et 9 sont obtenues à partir de réseaux dont les listes de préférences sont produites par la construction de Erdős-Rényi : tout noeud Y a probabilité p d'être présente dans la liste de préférence de X . La marque $\sigma(X, Y)$ est alors choisie aléatoirement. On note expérimentalement que le temps de convergence ne dépend pas de p . On se place donc par la suite dans le cas où $p = 1$.

Algorithm 1 Calcul b-couplage stable statique

procedure STATIQUE(r, b) $\triangleright$ Un réseau non couplé r , et $b \in \mathbb{N}^*$ **while** *Il existe une paire bloquante* **do** Soit $\{n, p\}$ une paire bloquante prise aléatoirement Si nécessaire, découpler le pire partenaire de n (resp. p) Coupler n et p **end while****end procedure**

L'exécution de *STATIQUE* permet de produire la figure 3.(a). Notons que l'implémentation pratique avait pour objectif d'exposer grossièrement les dépendances de chaque variable et était peu optimisée. Par suite, une meilleure complexité serait possible.

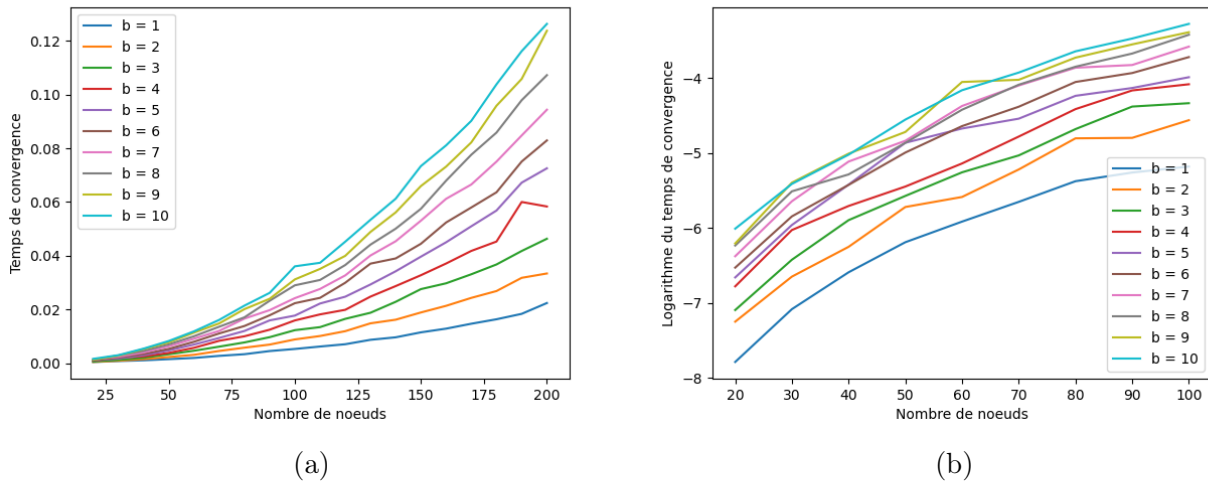


FIGURE 3 – Temps de convergence linéaire a et logarithmique b de *STATIQUE*

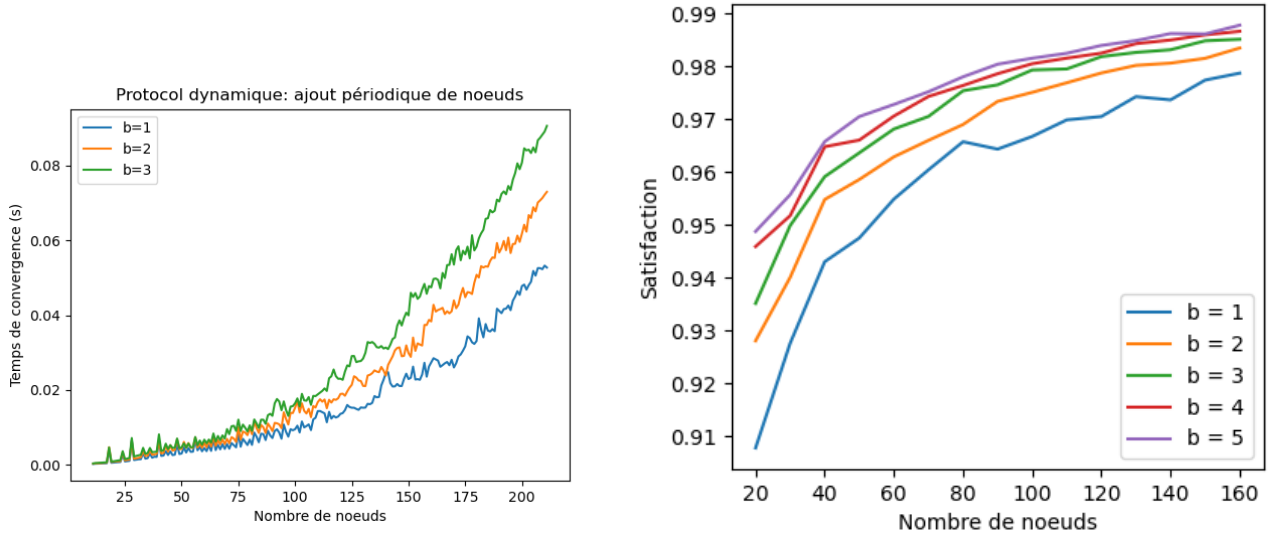
D'après l'allure de cette courbe, on s'attend que la complexité de l'algorithme soit de l'ordre de $O(2^{n+b})$, ce qui est mis en évidence par la courbe logarithmique de la figure 3.(b), dont le profil est proche de celui d'une fonction linéaire.

2.2 Adaptation dynamique naïve de l'algorithme de calcul

On s'intéresse ici à une adaptation naïve de l'algorithme précédant, consistant à itérer *STATIQUE* depuis un réseau de couplage vide à chaque modification du réseau. Par correction de *STATIQUE*, on garantit donc la conservation d'un couplage stable après convergence. On trace en figure 4.(a) les temps de convergence après ajout successifs de 200 noeuds à partir d'un réseau de 10 noeuds. Comme attendu, les profils de temps de convergence sont similaires à ceux obtenus en 3.(a).

2.3 Adaptation dynamique de l'algorithme de calcul par modification locale

Lors de l'ajout d'un noeud n , on peut espérer que seuls les noeuds de plus grande marque selon n modifieront leur couplage. Idem, lors du retrait d'un noeud n , les couplages des noeuds couplés à n seront *a priori* les seuls à être directement modifiés. Ainsi, la modification du réseau est une opération locale à la suite de laquelle on peut proposer une modification seulement locale



(a) Ajouts successifs de noeuds à partir de $|R| = 10$

(b) Satisfaction via *STATIQUE*

FIGURE 4

du couplage dans l'optique de "réparer" la perturbation occasionnée. J'ai donc implémenté une adaptation de *STATIQUE* dans laquelle le retrait d'un noeud n entraîne la suppression

- des arêtes $\{n, p\}$ dans le couplage, et pour chaque tel p ,
- des arêtes $\{q, r\}$ si $\sigma(q, r) < \sigma(q, p)$ ou $\sigma(q, r) < \sigma(r, p)$. On dira que $\{q, r\}$ est une *paire bloquante de profondeur 2*

En effet, pour un tel p , les couples $\{q, p\}$ ou $\{r, p\}$ seront des paires bloquantes et nuisent à la stabilité du couplage. On peut continuer ce processus d'élimination de couples en appliquant la deuxième étape à $p = q$ et $p = r$, et ainsi de suite en recensant à chaque nouvelle étape i les *paires bloquantes de profondeur $i + 1$* . Il suffit de s'arrêter lorsqu'aucun couple n'est modifié. On garantit de cette manière de retrouver toutes les paires bloquantes du réseau. Enfin, on relance *STATIQUE* sur le réseau restant pour reconstruire un couplage stable sur la base de l'ancien.

Dans le cas d'un ajout d'un noeud n , on procède à partir du 2^{ème} tiret pour $p = n$.

Le processus de retrait de couples reste coûteux : on parcourt plusieurs fois le couplage et plus il y a de paires bloquantes recensées, plus l'exécution de *STATIQUE* sera longue. Dans une démarche simplificatrice, on ne retire que les couples de profondeur 2 : on risque alors de manquer des paires bloquantes et de finalement trouver un couplage non stable. Pour quantifier la qualité du couplage calculé -c'est-à-dire à quel point celui-ci se rapproche d'un couplage stable-, on introduit la *fonction de satisfaction* S [4] :

$$S = \frac{1}{|R|} \sum_{i \in R} S_i = \frac{1}{|R|} \sum_{i \in R} \left(\frac{c_i}{b} + \sum_j \frac{R_j - C_j}{bL_i} \right) \quad (6)$$

où, pour i et j deux noeuds, c_i désigne le nombre de couples dans lesquels i intervient, R_j le rang de j dans $Pref(i)$, C_j le rang de j parmi les noeuds couplés avec i et $L_i = |Pref(i)|$.

Intuitivement, plus S_i est proche de 1, plus i est satisfait par son couplage. Moyenner les S_i permet donc de mesurer la satisfaction générale du réseau, qui est un réel dans $[0, 1]$. La figure 2.(b) permet de confirmer cette intuition : la satisfaction obtenue à l'issue de *STATIQUE* présente de hauts taux de satisfaction pour $|R| \geq 50$.

Comme montré en figure 5, l'implémentation de cette nouvelle adaptation par modification locale pour le cas d'ajouts successifs de noeuds converge sensiblement plus rapidement que dans le cas naïf. La courbe du temps de convergence s'est affaissée en un profil linéaire. De surcroît, le couplage ainsi déterminé est de haute satisfaction si $|R| \geq 50$ (> 0.95). Ces valeurs

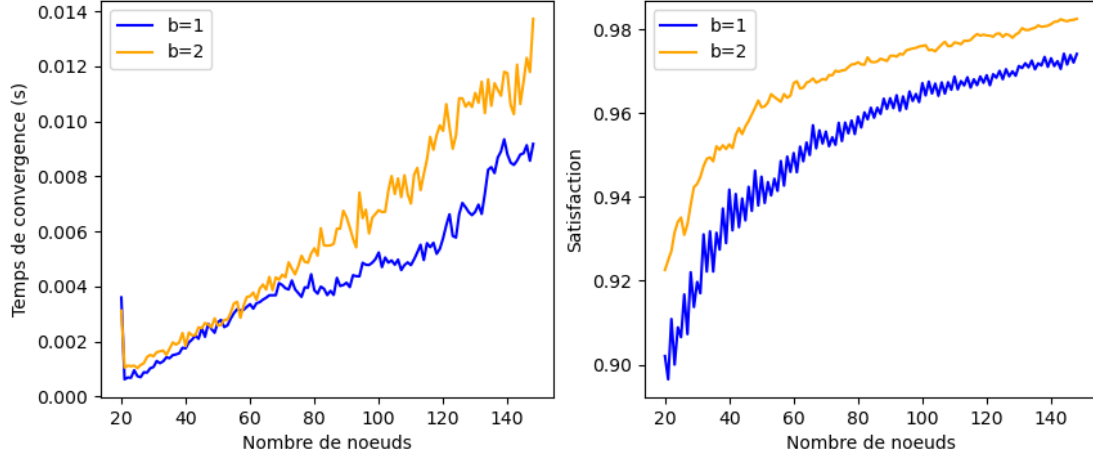


FIGURE 5 – Calcul par modification locale pour l’ajout successif de 150 noeuds à partir d’un réseau de cardinal $|R| = 20$

sont à comparer avec celles obtenues en Section 2.3 : un couplage non stable est de satisfaction aléatoire, d’espérance $\frac{1}{2}$.

En figure 6, on considère l’adaptation dans le cas d’ajouts et retraits successifs. Au fur et à mesure des ajouts et retraits, chaque liaison est susceptible d’être supprimée. J’ai donc mesuré en figure 7 le nombre de tours (d’ajouts ou de retraits) que dure en moyenne un couple. Cette donnée permet d’évaluer le temps dont dispose chaque couple de noeuds pour opérer leur transfert de données via cette liaison. Par exemple, pour un réseau réel de 300 utilisateurs connectés où chaque utilisateur reste connecté en moyenne 10 minutes, un noeud est ajouté/retiré chaque seconde. Si, comme la figure 7 le laisse entendre, chaque liaison reste active pendant 30 tours, alors les noeuds disposent de 30 secondes pour échanger leurs données. Cette estimation numérique est grande devant le temps de convergence de l’algorithme. Du point de vue très simplificateur de ce modèle de réseau, cette méthode de modification locale du couplage paraît donc être adaptée à une utilisation dans le cadre de réseaux pair à pair.

3 S’affranchir de la condition d’acyclicité

Comme expliqué en introduction, un b -couplage stable n’existe pas toujours, notamment si le graphe de préférence est cyclique. Dans l’exemple suivant, on représente en rouge un 1-couplage, mais il n’existe pas de couplage stable.

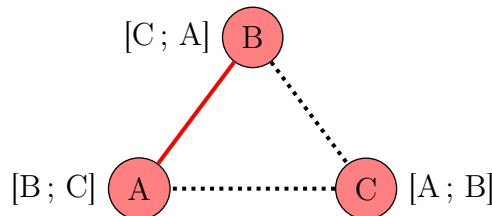


FIGURE 8 – On reprend le formalisme de la figure 2

J’ai essayé d’adapter *STATIQUE* au cas de graphes de préférences cycliques en détruisant une arête dès qu’un cycle était détecté, puis un noeud si sa liste de préférence a été complètement vidée. Les résultats de convergence en figure 9 sont peu encourageants : non seulement

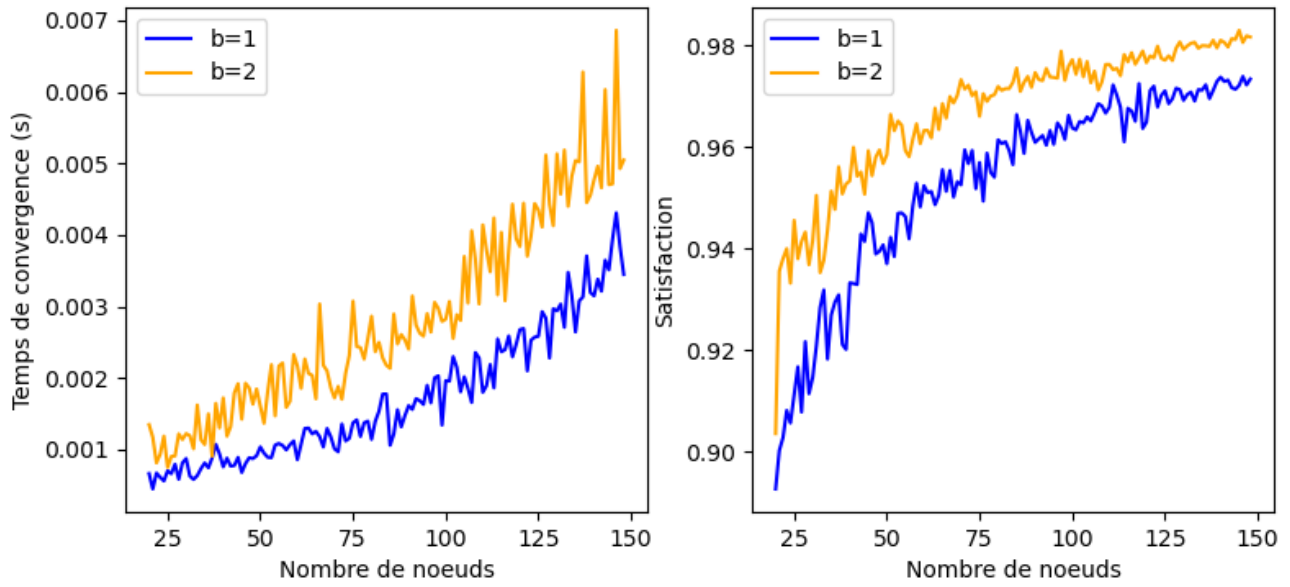


FIGURE 6 – Calcul par modification locale pour l’ajout et le retrait successif de 40 noeuds à partir d’un réseau de cardinal fixe $|R| = n \pm 1$

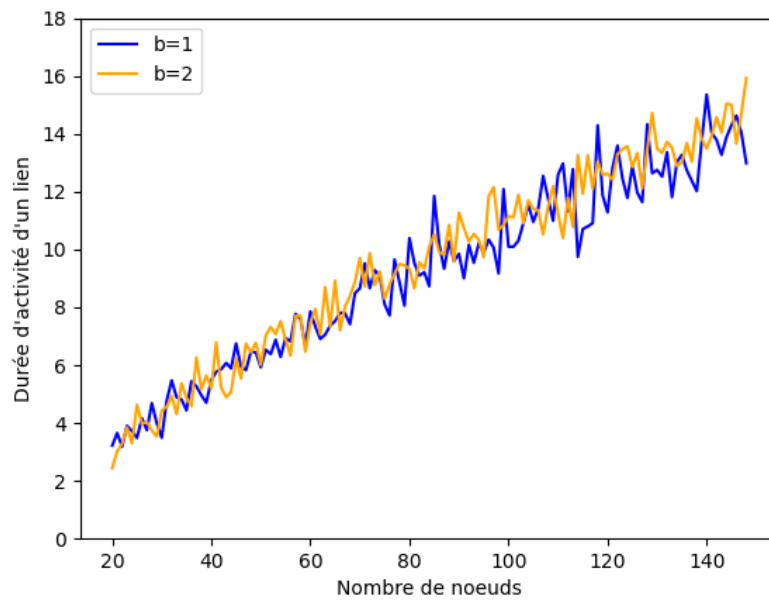


FIGURE 7 – Durée de maintien de chaque liaison (en nombre de modifications du réseau)

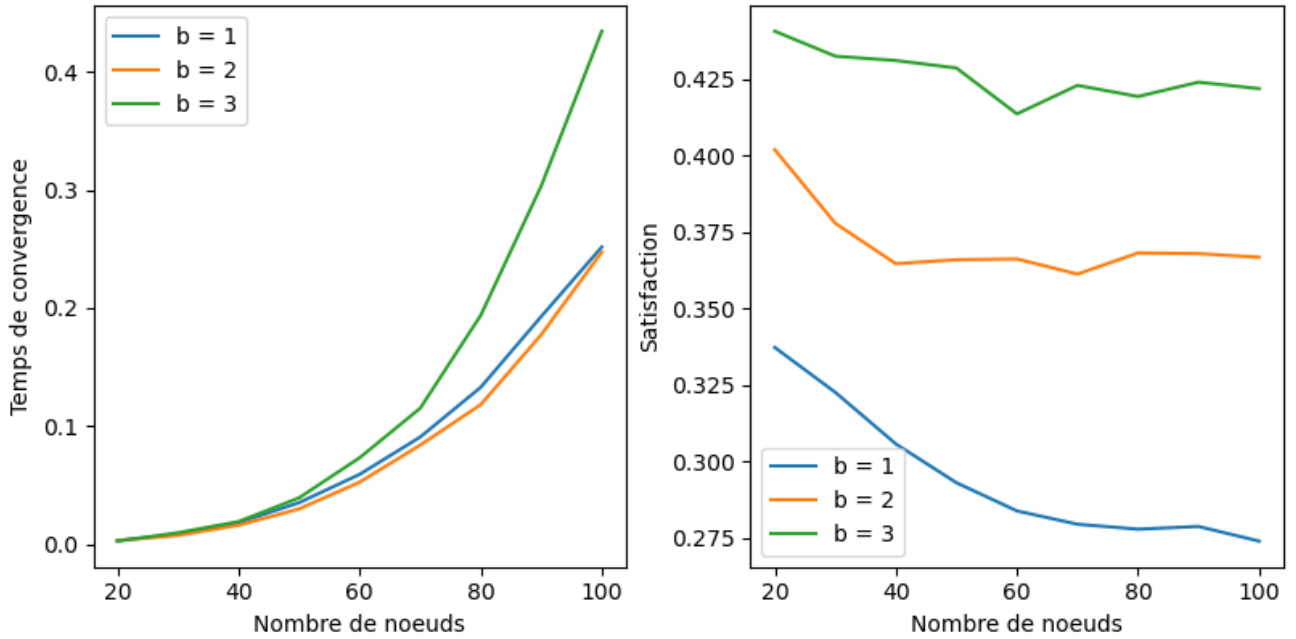


FIGURE 9 – Temps de convergence et satisfaction après *CYCLIQUE*

l'algorithme converge plusieurs centaines de fois plus lentement que *STATIQUE*, mais la satisfaction est très basse. En l'état, cette adaptation n'a pas d'utilité pratique.

4 Conclusions et perspectives

L'implémentation des réseaux pair à pair à la lumière de la théorie des couplages stables est un domaine riche et prometteur, permettant d'optimiser les liaisons faites en fonction de préférences de chaque utilisateur. Cela n'est cependant possible que dans le cas de préférences donnant naissance à un graphe de préférences acyclique, condition dont il n'est pas simple de s'affranchir. A ce sujet, il serait donc intéressant d'approfondir les raisons de l'échec de l'algorithme d'élimination de cycles et d'en déduire une nouvelle adaptation plus pertinente.

Dans le cadre de la forte simplification qui est faite ici du protocole de couplage mis en place par les réseaux pair à pair, le couplage par b-couplages stables semble converger suffisamment rapidement pour le partage de données entre utilisateurs. Il serait intéressant d'implémenter un algorithme de couplage couramment utilisé dans les réseaux pair à pair afin de pouvoir comparer sa performance à celle de mon implémentation.

Il ressort de mon étude que la satisfaction semble être une mesure plus adéquate que celle de stabilité pour quantifier la qualité d'un couplage. En effet, la stabilité est une notion binaire, peu adaptée à un problème d'optimisation et à la recherche d'un facteur d'approximation. La satisfaction étant un nombre entre 0 et 1, elle est pour sa part adaptée à un tel problème. De surcroît, l'algorithme d'approximation d'un couplage de satisfaction maximale proposé par G. Georgiadis et F. Papatriantafilou (2007) [4] est fonctionnel dans le cas de graphes de préférences cycliques.

5 Annexes

5.1 Preuve de terminaison de *STATIQUE*

La preuve de terminaison de l'algorithme repose sur la notion de *paire aimante*. Une paire $\{n, p\}$ est dite aimante si n et p sont chacun premier dans la liste de préférence de l'autre. En

d'autres termes, il s'agit d'un 2-cycle dans le graphe de préférence.

Une paire aimante existe toujours dans le réseau si le graphe de préférence est acyclique. Cela s'obtient en effectuant un parcours en profondeur sur le graphe de préférence : par acyclicité de celui-ci, on ne traverse jamais deux fois le même noeud (hormis dans le cas d'un 2-cycle). Le dernier noeud traversé par le parcours possède néanmoins une arête, qui pointe donc nécessairement vers son parent : on a trouvé une paire aimante. Si cette paire aimante ne fait pas partie d'un couplage, c'est alors une paire bloquante.

Théorème : L'algorithme 1 termine.

Démonstration : Par soucis de concision, on montre le théorème dans le cas simple où $b = 1$. On montre par récurrence sur le nombre N de noeuds intervenant dans une paire bloquante que la boucle *while* termine :

- Les cas $N = 0$ et $N = 1$ sont triviaux : le couplage est déjà stable.
- On suppose le résultat pour $N - 2$. D'après ce qui précède, R contient une paire aimante. De manière probabiliste, la boucle *while* finit par choisir cette paire aimante $\{n, p\}$, auquel cas le couple ainsi formé est alors inséparable. En effet, n et p ne font plus partie d'une paire bloquante, et si une paire bloquante $\{n, r\}$ venait à être produite à un moment ultérieur, c'est que $\sigma(n, r) > \sigma(n, p)$ ce qui est absurde. Le nombre de noeuds intervenant dans une paire bloquante décroît donc de 2 : par hypothèse de récurrence, la boucle termine.

□

En plus de permettre de prouver la terminaison de *STATIQUE*, la notion de paire aimante est centrale au fonctionnement de cet algorithme. En effet, lorsque deux noeuds sont couplés via une paire aimante, chacun est retiré de la liste de préférence de l'autre, ce qui permet de trouver une nouvelle paire aimante au sein du graphe de préférences. On continue tant qu'il existe une paire aimante.

5.2 Code principal en Ocaml

Le code entier est très long. Les types et les fonctions principales de *STATIQUE* sont listées ci-dessous. D'autres fonctions auxiliaires sont utilisées :

- `casse_match n p` supprime le couple $\{n, p\}$.
- `choix_best r` choisi aléatoirement un noeud et son pair préféré dans R .
- `check_loving n p` vérifie si $\{n, p\}$ est une paire aimante, et couple $\{n, p\}$ en fonction.

```
type trileen = Unmatched | Unloving | Loving
(*Différents états d'un noeud au sens d'une configuration / d'un couplage*)

type node = {
  id: int;
  mutable ind_noeuds : int;
  (*indice du node dans le tableau reseau.noeuds *)

  mutable ind_unloving : int;
  (*indice du node dans reseau.unloving*)

  config : pfile;
  (* Tableau de noeuds rangé par marque décroissante des nodes
  avec lesquels le node peut échanger des données. C'est la liste
```

d'adjacence de node dans le graphe d'acceptance. Les cases non utilisées sont à la fin du tableau et à None*)

```
couplage : pfile;
  (*Tableau de taille b ordonné par marque décroissante des nodes intervenant
  dans le couplage*)

mutable num_loving : int;
  (*Nombre (majoré par b et par couplage.len) de peers au sein de couplage
  qui sont loving*)
}
and pfile = {
  (*Tableau trié selon la marque décroissante. Les cases non utilisées sont à None*)
  mutable tab : champ option array;
  mutable len : int;
}
and champ = {
  (*Structure modélisant une arête depuis un noeud*)
  n : node;
  e : trileen;
  (*indique si node est dans le couplage ou pas, relevant seulement pour config*)

  marque : int;
}

type reseau = {
  mutable noeuds : node option array;
  (*tableau des nodes dans le réseau. Le node d'indice node.id est à
  l'indice node.id*)

  mutable len_noeuds : int;
  (*Taille de noeuds*)

  mutable len_unloving : int;
  (*Taille de unloving. Le protocol s'arrête quand ça atteint 0*)

  mutable unloving : node option array;
  (*tableau des noeuds dont le couplage n'est pas composé que de peers loving.
  La structure est la même que noeuds*)
}

let couplement node peer reseau marque =
(*On a trouvé une paire devant être couplée:
  On ajoute la paire au couplage et
  si la paire est loving on retire de chaque config le pair correspondant et
  on les enlève du unloving du réseau et on réduit le unloving du reseau si
  ils sont complètement couplés avec des loving*)
pfile_insere node.couplage peer Unloving marque;
```

```

pfile_inseere peer.couplage node Unloving marque;
set_flag_in_pfile node.config peer Unloving None;
set_flag_in_pfile peer.config node Unloving None;
match peer.config.tab.(peer.config.len -1) with
| None -> (print_string "\nLe peer choisi n'a pas de config\n"; exit(1))
| Some c when c.n.id != node.id ->
    (*La paire n'est pas loving*)
    ()
| Some c ->
    traitement_loving node peer reseau

```

```

let traitement_loving node peer reseau =
(*On effectue les démarches nécessaires pour coupler définitivement node et peer*)
node.num_loving <- node.num_loving +1;
peer.num_loving <- peer.num_loving +1;
set_flag_in_pfile node.couplage peer Loving None;
set_flag_in_pfile peer.couplage node Loving None;
if node.num_loving = !b then begin
    (*Le node est complètement couplé qu'avec des paires incassables:
    on le retire de unloving*)
    nullify_unloving reseau node;
    remove_from_couplages reseau node;
end;
if peer.num_loving = !b then begin
    nullify_unloving reseau peer;
    remove_from_couplages reseau peer;
end;
rm_config node peer;
rm_config peer node

```

```

let proposal node peer reseau m =
let wno = worst node in
let wpo = worst peer in
if node.couplage.len = !b && peer.couplage.len = !b then
    match wno, wpo with
    | Some wn, Some wp when (m, m) >= (wp.marque, wn.marque) ->(
        (*On s'assure que faire un changement garantit de trouver un meilleur état*)
        (*On retire les worst mate de peer et node pour les remplacer avec un autre
        mate de marque str. sup*)
        casse_match wn.n node;
        casse_match wp.n peer;
        remove_nth peer.couplage 0;
        set_flag_in_pfile peer.config wp.n Unmatched None;
        remove_nth node.couplage 0;
        set_flag_in_pfile node.config wn.n Unmatched None;
        complement node peer reseau m)
    | Some n, None | None, Some n -> failwith "Broken :/"
    | None, None -> failwith "Broken :/"

```

```

| _ -> ()
else if node.couplage.len = !b && marque wno <= m then
  match wno with
  | Some wn ->(
    assert(node.num_loving < !b);
    casse_match wn.n node;
    (* On enlève le worst du couplage de node pour avoir la place pour peer
      (qui est mieux)*)
    remove_nth node.couplage 0;
    set_flag_in_pfile node.config wn.n Unmatched None;
    complement node peer reseau m)
  | None -> failwith "Broken :/"
else if peer.couplage.len = !b && marque wpo <= m then
  match wpo with
  | Some wp ->(
    assert(peer.num_loving < !b);
    casse_match wp.n peer;
    (* On enlève le worst du couplage de node pour avoir la place pour peer
      (qui est mieux)*)
    remove_nth peer.couplage 0;
    set_flag_in_pfile peer.config wp.n Unmatched None;
    complement node peer reseau m)
  | None -> failwith "Broken :/"
else if peer.couplage.len != !b && node.couplage.len != !b then( (*Ajout aux deux*)
  complement node peer reseau m
)
else ()

```

```

let rec initiative_best reseau = (*Réalise une initiative via stratégie best mate*)
  let node = choix_best reseau in
  assert(not (has_before node.config node 0));
  if node.config.len > 0 then begin
    match node.config.tab.(node.config.len -1) with
    | None -> failwith "Erreur: le best mate du node est à None"
    | Some best -> begin
      if best.n.ind_unloving = -1 || best.n.ind_noeuds = -1 then begin
        (* On retire de la config et on repioche. Le retrait d'un node cause
          au plus len_config repiochages -> ok complexité
          Ceci arrive quand un best couplé apparaît dans la config de node *)
        let _ = pfile_defile node.config in
          ()
        end
      else if best.e != Unmatched then begin
        assert(mem_couplage node best.n);
        check_loving node best.n reseau
      end
      else begin
        (* On voit s'il est intéressant de coupler node et best
          -> c'est intéressant pour node, l'est-ce pour best?*)

```

```

        proposal node best.n reseau best.marque
    end
end
end

else (
    if !b >= reseau.len_unloving then reseau.len_unloving <- 0
    else exit(1))

let protocol reseau =
    num.num <- 0;
    while reseau.len_unloving > 0 do
        initiative_best reseau;
    done

let connections_init p reseau = (*Génère une liste de préférence selon erdos-renyi*)
    let pf ={
        tab = Array.make max_config None;
        len = 0
    } in
    for i = 0 to reseau.len_unloving -1 do
        match reseau.unloving.(i) with
        | None -> failwith "None au milieu du unloving"
        | Some node ->
            let max = (2 lsl 29) -1 in
            let n = randint max in
            let m =
                if float_of_int n < p *. (float_of_int max) then
                    randint max_marque
                else
                    0
            in
            pfile_insere pf node Unmatched m;
    done;
    pf

```

Références

- [1] B. Cohen. The bittorrent protocol specification. *BitTorrent.org : BEP 3*, 2008.
- [2] A.-T. Gai et al. Acyclic preference systems in p2p networks. In *European Conference on Parallel Processing*, pages 825–834, Berlin, Heidelberg, 2007.
- [3] D. Lebedev et al. On using matching theory to understand p2p network design. *International Network Optimization Conference*, 2007.

- [4] M. Papatriantafylou G. Georgiadis. Adaptive distributed b-matching in overlays with preferences. In *International Symposium on Experimental Algorithms*, volume 1, pages 104–121, Berlin, Heidelberg, 2012.
- [5] F. Mathieu. Self-stabilization in preference-based systems. *Discrete Applied Mathematics*, 2007.